

# PROJECT ARJUNA: INTERCEPTOR AI MODULE

*Autonomous Target Acquisition, Hybrid Tracking, and FPV Interception System*  
*Comprehensive Project Evolution, AI Model Enhancements, and Edge/Radxa Hardware Deployment Report*

|                                                        |                                                               |
|--------------------------------------------------------|---------------------------------------------------------------|
| <b>System Name:</b> Arjuna GCS / Interceptor AI        | <b>Deployment Scope:</b> Ground Station & Edge SBC (Radxa)    |
| <b>Architecture:</b> Hybrid YOLO + CSRT/KCF + Servoing | <b>Document Scope:</b> Full Technical Progression & Solutions |

## 1. Executive Summary & Mission Objective

The Interceptor AI Module (Arjuna Platform) is a precision autonomous targeting, visual servoing, and flight guidance software suite developed for Counter-Unmanned Aerial Systems (C-UAS) and high-speed FPV interceptor drones. The system pairs real-time deep neural network object detection with classical computer vision tracking and closed-loop flight controller telemetry to achieve autonomous target lock, continuous optical tracking, trajectory prediction, and dynamic yaw/pitch/roll visual servoing.

A primary architectural mandate throughout the development lifecycle has been dual-tier deployment capability: running seamlessly on high-performance ground control workstations with dedicated NVIDIA GPUs while maintaining extreme efficiency, stability, and low latency on embedded edge single-board computers (specifically the Radxa SBC family and companion microcomputers).

## 2. Chronological Project Evolution & Development History

The repository has progressed through five distinct development phases from an initial monolithic script to a robust modular Ground Control Station:

| Milestone / Commit                                     | Key Architectural Changes                                                                                                                           | Operational Impact                                                                                                    |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Phase 1: Inception</b><br>(e4abbd9, 884fd1d)        | Initial monolithic main.py, YOLOv8n detector, CSRT/KCF OpenCV tracking, baseline MSP SET_RAW_RC @ 50Hz, custom training scripts.                    | Established baseline visual servoing proof-of-concept on INAV flight controllers over USB-serial link.                |
| <b>Phase 2: Modular Core</b><br>(aaf7ae5)              | Decoupled architecture: interfaces, plugin managers, Kalman filters, motion predictors, distance estimators, failsafe manager, and threaded worker. | Eliminated monolithic bottlenecks; enabled dynamic plugin swapping and mathematically rigorous 4-axis flight control. |
| <b>Phase 3: Arjuna GCS</b><br>(c6b02a0)                | PyQt6 Arjuna GCS: multi-page shell, SQLite target database, live feed HUD overlay, telemetry grapher, PID tuning panel, structured logger.          | Delivered professional operator workstation with persistent target profiling and real-time flight diagnostics.        |
| <b>Phase 4: Standalone Build</b><br>(a2518fd)          | Centralized paths.py for AppData persistence; PyInstaller compilation pipeline (build_exe.py); MAVLink flight controller plugin.                    | Enabled standalone single-click executable distribution without manual Python environment setup on target systems.    |
| <b>Phase 5: Control Updates</b><br>(54deafc, 92f6400+) | Real-time keyboard overrides, multi-channel AUX mapping (Arm CH5, Mode CH6), throttle stepping, scale-aware tracking engine.                        | Improved pilot safety, multi-mode drone flight control, and robust tracking in aggressive flight maneuvers.           |

## 3. Core Technical Architecture & Subsystem Deep Dive

- Multi-Threaded Tracking Worker (TrackingWorkerThread):** Decouples video capture and computer vision processing from the UI rendering thread. Emits processed video frames, detected bounding boxes, target profiles, and FPS metrics at up to 60+ Hz via PyQt signals.
- Hybrid Detection & Tracking Engine:** Employs a multi-tier tracking hierarchy: (1) Scale-Aware Lock for ground-truth bounding box size, (2) Optical Flow (Lucas-Kanade) / CSRT / KCF for micro-motion tracking between neural frames, and (3) Periodic YOLO verification (YOLO\_EVERY\_N) to correct optical drift and reacquire lost targets.

- **4-Axis Visual Servoing Controller (FPVFollowController):** Transforms 2D normalized screen error (X, Y) into proportional, integral, and derivative (PID) stick commands for Yaw, Pitch, Roll, and Throttle. Incorporates soft deadband bleeding, exponential curves, output slew limiting, and lead-time forward projection.
- **Dual Flight Controller Abstraction:** Provides native support for both INAV / Cleanflight / Betaflight (via MSP protocol SET\_RAW\_RC at 50Hz) and ArduPilot / PX4 (via MAVLink v1/v2 RC channel overrides and telemetry heartbeats).
- **Target Profile & Database System (TargetStore):** Persistent SQLite repository storing target profiles, threat classification, visual feature templates, estimated physical dimensions, and engagement histories.
- **Comprehensive Safety & Failsafe Subsystem:** Monitors target lock stability, telemetry heartbeat timeouts, extreme vehicle attitude angles, and invalid bounding boxes, immediately executing disarm or flight mode fallback when thresholds are breached.

## 4. Problems Solved for the Main AI Model

- **Problem 1: Latency & Frame Rate Drop on Deep Models:** Deep neural networks introduce 15–40ms latency per frame. Solved by decoupling the pipeline: YOLO runs periodically (every 6–12 frames), while high-speed correlation filters (CSRT/KCF) and optical flow track intermediate frames at 60+ FPS. A velocity-based motion predictor projects the target position forward by a lead time (lead\_s = 0.12s), eliminating control latency.
- **Problem 2: Target Loss During Rapid Turns and Occlusions:** Optical trackers drift when targets maneuver sharply. Solved by implementing an automated reacquisition engine with IoU spatial matching and histogram similarity validation. The system enters a 'Hold' state for up to 20 frames while YOLO scans the predicted trajectory window for re-locking.
- **Problem 3: Generalization Across Unknown Aerial Threats:** Standard COCO models only recognize generic classes. Solved by supporting triple-mode detection: 'world' for zero-shot open-vocabulary prompts (YOLO-World), 'custom' for fine-tuned weights (drone\_missile\_best.pt), and 'coco' for lightweight baselines.
- **Problem 4: Bounding Box Jitter and Distance Estimation Instability:** Minor box fluctuations produce severe noise in pinhole distance estimation. Solved by implementing ScaleAwareLock and PixelLockEngine to decouple center tracking from scale estimation, applying exponential moving average filtering on pixel dimensions.

## 5. Problems Solved for Radxa & Embedded Edge Hardware

- **Problem 1: Extreme CPU/GPU Compute Constraints:** Edge SBCs lack high-end GPUs. Solved by implementing configurable inference throttling (YOLO\_EVERY\_N), fallback KCF correlation trackers, normalized coordinate math (allowing 640x480 resolution without losing accuracy), and dedicated --cpu and --safe-mode flags.
- **Problem 2: Complex Native Dependencies on SBC Environments:** Setting up PyTorch, CUDA, OpenCV, and PyQt6 on Linux/Windows SBCs is error-prone. Solved by creating the build\_exe.py PyInstaller pipeline, compiling all hidden imports, models, and calibration assets into a single standalone distribution.
- **Problem 3: Video Ingestion Locks & Driver Incompatibilities:** DirectShow capture on SBC USB capture cards frequently hung GUI threads. Solved by disabling MSMF hardware transforms, multi-backend fallback probing, and adding automated synthetic test frames on capture loss.
- **Problem 4: Headless Onboard Execution:** In-air Radxa boards operate without displays. Solved by building a lightweight headless CLI mode (--cli) that executes the complete tracking and MSP/MAVLink guidance loop using under 180MB RAM.
- **Problem 5: Data Persistence in Portable Executables:** Executables extract to temporary directories (sys.\_MEIPASS), losing configs on exit. Solved by introducing paths.py to isolate read-only assets from persistent user data stored in standard OS directories (%APPDATA%\ArjunaGCS).

## 6. Visual Servoing, PID Guidance, & Flight Controller Link

| Axis / Channel    | Default PID (Kp / Ki / Kd) | Max Limit           | Behavior / Function                                         |
|-------------------|----------------------------|---------------------|-------------------------------------------------------------|
| Yaw (CH4)         | 340.0 / 45.0 / 60.0        | ±400 µs (1100–1900) | Steers drone heading to align target with optical center.   |
| Pitch / Alt (CH3) | 310.0 / 40.0 / 55.0        | ±360 µs (1140–1860) | Adjusts nose pitch / elevation to match target height.      |
| Roll (CH1)        | 80.0 / 8.0 / 15.0          | ±120 µs (1380–1620) | Optional coordinated bank/strafe during high-rate turns.    |
| Throttle (CH2)    | Stepped / Manual (±25 µs)  | 1000–2000 µs        | Manual operator throttle or distance-based approach pacing. |

## 7. Arjuna Ground Control Station (GCS) Interface Overview

- **Live Feed HUD:** Displays artificial horizon line, dynamic lock reticle, lead prediction crosshairs, target bounding box, telemetry overlay (altitude, battery, distance, heading), and flight mode status.
- **Target Database Page:** SQLite-backed library of targets with threat categorization, physical dimension setup, and engagement histories.
- **Distance Calibration Wizard:** Interactive tool to calculate camera focal length based on known object width/height at set distances, saving to calibration.json.
- **PID Tuning Panel:** Live adjustment of PID gains, deadzones, lead compensation, and stick invert toggles with instant JSON sync.
- **Joystick & Gamepad Page:** Direct mapping of USB gamepads, RC transmitters, button binds (Lock, Arm, Mode, Disarm), and dead-man safety overrides.
- **System Logs & Telemetry Page:** Categorized, searchable event logs (Drone, Vision, Safety, System) with exportable telemetry logging.

## 8. Key Conclusions & Presentation Takeaways

1. **Proven Hybrid AI Architecture:** Successfully pairs the accuracy of deep learning with the low compute footprint and 60+ FPS speed of correlation tracking.
2. **Demonstrated Edge Readiness:** Specifically engineered to overcome the compute, thermal, and memory limits of Radxa SBCs without compromising real-time tracking fidelity.
3. **Universal Flight Controller Compatibility:** Seamless interoperability with both INAV (MSP) and ArduPilot/PX4 (MAVLink) ecosystems.
4. **Production-Grade Packaging:** Turnkey standalone executable distribution with persistent user configuration ready for field deployment.